

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	SHAFINA.A	Reg.No.:	192521474
Department:	B.TECH(IT)	Date:	25/07/26

ANNEXURE A

1. A Booth's multiplication program produces an incorrect product for the inputs -13×9 . Debug the algorithm by identifying the incorrect shift or addition/subtraction step and optimize the implementation to reduce execution cycles.

For **Booth's multiplication**, the correct result is:

$$-13 \times 9 = -117$$

If your program gives an incorrect product, the most common bug is in the **arithmetic right shift** or in the **add/subtract decision based on Q_0Q_{-1}** .

Correct Booth algorithm for -13×9

Use **5-bit signed representation**:

- $M = -13 = 10011$
- $-M = +13 = 01101$
- $Q = 9 = 01001$
- $A = 00000$
- $Q_{-1} = 0$

Step	Q_0Q_{-1}	Operation	After Arithmetic Right Shift
Initial	—	$A = 00000, Q = 01001, Q_{-1} = 0$	—
1	10	$A = A - M$	$A = 00110, Q = 10100, Q_{-1} = 1$
2	01	$A = A + M$	$A = 11100, Q = 11010, Q_{-1} = 0$
3	00	No operation	$A = 11110, Q = 01101, Q_{-1} = 0$
4	10	$A = A - M$	$A = 00000, Q = 00110, Q_{-1} = 1$
5	01	$A = A + M$	$A = 11001, Q = 00011, Q_{-1} = 0$

Final product:

$$AQ = 1100100011$$

This is a 10-bit two's complement number:

$$1100100011_2 = -117_{10}$$

Likely bug in your program

Check these lines in your implementation:

1. For $Q_0 Q_{-1} = 10 \rightarrow$ subtract M
 $A = A - M$
2. For $Q_0 Q_{-1} = 01 \rightarrow$ add M
 $A = A + M$
3. For 00 or $11 \rightarrow$ no operation
4. The shift must be arithmetic right shift, not logical right shift. The sign bit of A must be preserved.
5. The shift order must be:

$A, Q, Q_{-1} \rightarrow$ Arithmetic Right Shift

Specifically, the **old LSB of A becomes the new MSB of Q** , and the **old Q_0 becomes Q_{-1}** .

Optimization to reduce execution cycles

A basic Booth implementation performs **one iteration per multiplier bit**. To optimize:

- Use **radix-4 Booth multiplication**, which processes **2 multiplier bits per cycle**.
- Reduce the number of iterations from **5 to approximately 3** for a 5-bit operand.
- Precompute $M, 2M, -M$, and $-2M$ to reduce repeated arithmetic.
- Use fixed-width registers and direct bit operations instead of expensive conversions inside the loop.

Conclusion: For -13×9 , the expected result is -117 . If the output is wrong, first inspect the **arithmetic right shift**, especially preservation of the sign bit, and verify that the $10 \rightarrow$ subtract and $01 \rightarrow$ add cases are not reversed.

2. A processor implementing the **restoring division algorithm** generates an incorrect quotient for $29 \div 5$. Debug the iteration sequence, identify the faulty restore operation, and propose an optimized correction.

For **restoring division**:

$$29 \div 5 = 5 \text{ remainder } 4$$

Binary representation:

- Dividend $Q = 29 = 11101_2$
- Divisor $M = 5 = 00101_2$
- Initial $A = 00000$

Assuming a **5-bit restoring division algorithm**, the correct iteration sequence is:

Iteration	Shift A, Q	Subtract $A - M$	Decision	Final A, Q
1	00001,11010	11100	Negative $\rightarrow$ Restore	00001,11010
2	00011,10100	11110	Negative $\rightarrow$ Restore	00011,10100
3	00111,01000	00010	Positive $\rightarrow$ Keep	00010,01001
4	00100,10010	11111	Negative $\rightarrow$ Restore	00100,10010
5	01001,00100	00100	Positive $\rightarrow$ Keep	00100,00101

Therefore:

$$Q = 00101_2 = 5$$

$$A = 00100_2 = 4$$

Faulty restore operation

The likely bug is in **Iteration 1, 2, or 4**, where $A - M$ becomes negative.

The correct operation is:

$$A = A - M$$

if $A < 0$:

$$A = A + M \quad // \text{ Restore the previous } A$$

```

    Q0 = 0
else:
    Q0 = 1
A common faulty implementation is:
if A < 0:
    A = M    // WRONG
or:
if A < 0:
    A = A - M    // WRONG: subtracting again
The restore must undo the subtraction, so:

$$(A - M) + M = A$$


```

Optimized correction

Instead of physically performing a restore every time the subtraction is negative, use a **non-restoring division algorithm**. It avoids the immediate $A = A + M$ restoration and decides the next operation based on the sign of the partial remainder.

This reduces the number of add/subtract operations and improves execution efficiency.

Correct result:

$$29 \div 5 = 5 \text{ remainder } 4$$

Debug conclusion: The incorrect quotient is most likely caused by a faulty **restore after a negative partial remainder**. The restore should be $A = A + M$ followed by $Q0 = 0$. For optimization, replace restoring division with **non-restoring division** to reduce execution cycles

3. During the execution of the **non-restoring division algorithm**, the quotient bits are incorrect after the fourth iteration. Identify the logic error, correct the algorithm, and verify the final quotient and remainder.

For the **non-restoring division algorithm**, the key logic is that the **next operation depends on the sign of the current partial remainder**.

Assume the division is:

$$29 \div 5$$

Using 5-bit registers:

- Dividend $Q = 11101_2 = 29$
- Divisor $M = 00101_2 = 5$
- Initial $A = 00000$

Quotient: 3 full groups

Remainder: 2 objects left over

$$14 = 4 \times 3 + 2$$

3 full groups are the quotient, and 2 leftovers are the remainder.

Total objects to regroup

Total objects to regroup

Objects in each group

Objects in each group

Correct non-restoring logic

For each iteration:

1. If $A \geq 0$, perform:

$$A = 2A + Q_{\text{MSB}} - M$$

2. If $A < 0$, perform:

$$A = 2A + Q_{\text{MSB}} + M$$

3. Shift Q left.

4. If the **new** $A \geq 0$, set $Q_0 = 1$.

5. If the **new** $A < 0$, set $Q_0 = 0$.

Debugging the error

If quotient bits become incorrect **after the fourth iteration**, the likely logic error is:

The program is deciding the quotient bit using the **old sign of A** instead of the **new sign of A** after the add/subtract operation.

The correct decision is:

if $A \geq 0$:

$A = A - M$

else:

$A = A + M$

if $A \geq 0$:

$Q_0 = 1$

else:

$Q_0 = 0$

Another common error is **restoring A immediately** when it becomes negative. That converts the algorithm into restoring division and breaks the non-restoring quotient sequence.

Corrected algorithm

$A = 0$

$Q = \text{dividend}$

$M = \text{divisor}$

for $i = 1$ to n :

Shift left (A, Q)

if $A \geq 0$:

$A = A - M$

else:

$A = A + M$

if $A \geq 0$:

$Q_0 = 1$

else:

$Q_0 = 0$

if $A < 0$:

$A = A + M$

For $29 \div 5$, the final values are:

$Q = 00101_2 = 5$

$A = 00100_2 = 4$

Therefore:

$29 \div 5 = 5 \text{ remainder } 4$

Final diagnosis: The faulty quotient bits are caused by setting Q_0 based on the **wrong sign check** or by restoring the negative partial remainder too early. The quotient bit must be determined **after the current iteration's addition/subtraction**, and the final negative remainder is restored **only once, after all iterations**.

4. A Booth multiplier performs arithmetic right shifts as logical shifts, resulting in an incorrect signed multiplication result. Debug the issue and explain how arithmetic shifting improves correctness.

Correct Booth Algorithm Sequence

1. **Initialize**
 - $A = 0$
 - $Q = \text{Multiplier}$
 - $M = \text{Multiplicand}$
 - $Q_{-1} = 0$
2. **Check Q_0Q_{-1}**
 - $01 \rightarrow A = A + M$
 - $10 \rightarrow A = A - M$
 - $00 \rightarrow \text{No operation}$
 - $11 \rightarrow \text{No operation}$
3. **Perform Arithmetic Right Shift (ASR)**
 - Preserve the sign bit of A
 - Shift A, Q, Q_{-1} together
4. **Repeat for n iterations**
5. **Final Product**
 - Product = $[A, Q]$

Corrected NS / Pseudocode

$A = 0$

$Q = \text{Multiplier}$

$M = \text{Multiplicand}$

$Q_{-1} = 0$

for $i = 1$ to n :

if $Q_0 Q_{-1} == 01$:

$A = A + M$

else if $Q_0 Q_{-1} == 10$:

$A = A - M$

Arithmetic Right Shift (A, Q, Q_{-1})

Product = $[A, Q]$

Main correction: Replace the **Logical Right Shift (LSR)** with **Arithmetic Right Shift (ASR)**. The ASR copies the sign bit, preserving negative two's-complement values and producing the correct signed multiplication result.

IEEE 754 Floating-Point Addition – Debugging and Optimization

The error occurs because the **exponents of the two floating-point numbers are not aligned before adding their significands (mantissas)**.

For example:

$$X = M_1 \times 2^{E_1}, Y = M_2 \times 2^{E_2}$$

Before addition, both exponents must be made equal.

Correct Algorithm

Extract the **sign, exponent, and fraction** from both numbers.

Restore the hidden leading 1 for normalized numbers.

Compare the exponents.

Find the exponent difference:

$$D = |E_1 - E_2|$$

Right-shift the significand of the number with the **smaller exponent** by D positions.

Set both exponents to the larger exponent.

Add or subtract the significands depending on their signs.

Normalize the result.

Adjust the exponent.

Round the result using **Guard, Round, and Sticky (GRS) bits**.

Check for overflow or underflow.

Pack the final **Sign | Exponent | Fraction** fields.

Bug

Wrong:

$M1 + M2$

when:

$E1 \neq E2$

The program must first align the exponents.

Corrected Logic

if $E1 > E2$:

$\text{difference} = E1 - E2$

$M2 = M2 \gg \text{difference}$

$E = E1$

else:

$\text{difference} = E2 - E1$

$M1 = M1 \gg \text{difference}$

$E = E2$

Result = $M1 + M2$

Optimization

Instead of shifting the smaller significand **one bit at a time**, use a **barrel shifter** to perform the entire shift in one operation. Preserve **Guard, Round, and Sticky bits** during the shift to ensure correct IEEE 754 rounding.

Final answer: The bug is the failure to **align the exponents before significand addition**. The correction is to **right-shift the significand corresponding to the smaller exponent until both exponents match**. The process can be optimized using a **barrel shifter and GRS bits**, improving execution speed while maintaining IEEE 754 accuracy.
Identify the bug and optimize the alignment process

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation